

Backend & Authentication Document

How the server side of Raj Raani Collections is put together — the request lifecycle, every layer of the authentication system, and the one rule that keeps every secret out of the browser.

Brand	Raj Raani Collections
Runtime	Next.js 14 on Firebase App Hosting
Golden rule	API keys and secrets live only in server-side environment variables — never in frontend code

01

Backend architecture in one picture

There is no separate backend server to run — Firebase App Hosting runs the Next.js server itself, and that server is the only thing that ever holds a secret.

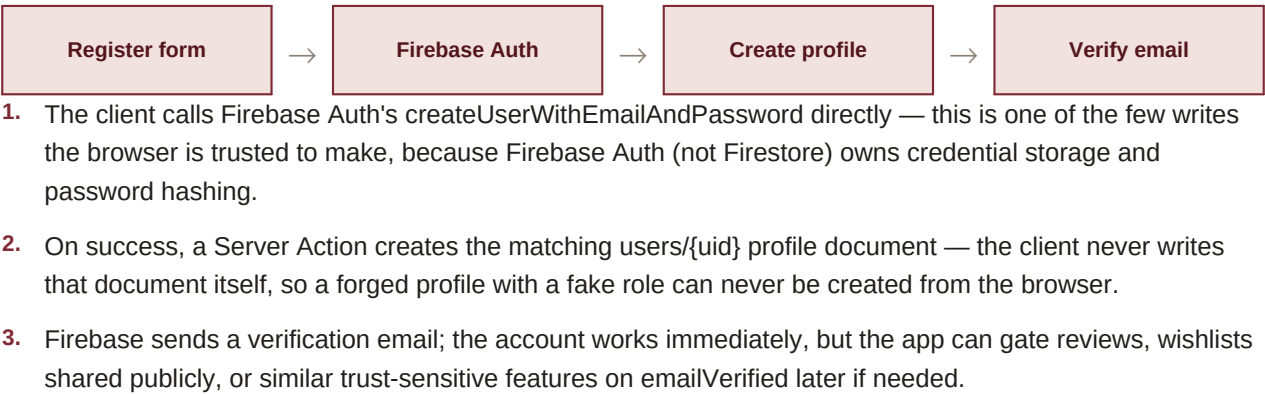


The rule that makes this safe

A file with 'use client' at the top runs in the browser — anything referenced there, including any environment variable, ships to every visitor and is readable via view-source. A file without it runs only on the server. Every secret in this document is read only from files in the second category.

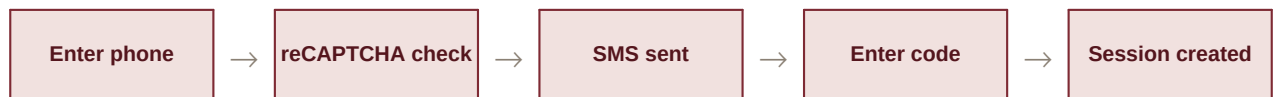
02

Sign-up — email & password



03

Sign-up & sign-in — phone OTP



1. The client requests a code via Firebase Auth's PhoneAuthProvider, protected by invisible reCAPTCHA (or App Check on supported platforms) so the SMS quota cannot be drained by a script.
2. Firebase sends a 6-digit code by SMS. The customer enters it in the same screen — no separate verify-otp route.
3. `confirmationResult.confirm(code)` completes sign-in entirely inside Firebase Auth; no OTP value or secret ever passes through the app's own server.
4. The same profile-creation Server Action from step 02 runs for a first-time phone sign-in.

Why phone first

Indian shoppers trust an OTP far more than an email link, and it removes "forgot my password" as a support burden entirely for anyone who signs up this way.

04

Sessions — cookies, not raw tokens

The Firebase client SDK issues a short-lived ID token, refreshed automatically in the browser. That token is never the thing middleware checks. Instead:

1. After a successful client-side sign-in, a Server Action calls the Firebase Admin SDK's `createSessionCookie(idToken)`, producing a long-lived, revocable session cookie.
2. That cookie is set **httpOnly, secure, sameSite=lax** — invisible to any JavaScript running on the page, which closes off the most common cookie-theft vector (XSS reading `document.cookie`).
3. Every request to a protected route runs through **middleware**, which calls `getAuth().verifySessionCookie(cookie, checkRevoked=true)` on the server — a valid-looking cookie that has been revoked is rejected, not just an expired one.
4. Sign-out clears the cookie **and** the client SDK session. For a compromised account, calling `revokeRefreshTokens(uid)` invalidates every session for that user immediately, everywhere, not just on the device that reported the problem.

05

The admin role

Admin is not a field a document can carry — it is a **custom claim** baked into the session cookie itself, set once via the Admin SDK and checked on every request.

```
// scripts/make-admin.js -- run once, from a trusted machine, then remove or lock down
const admin = require('firebase-admin');
admin.initializeApp({ credential:
  admin.credential.cert(require('./service-account.json')) });

admin.auth().setCustomUserClaims(OWNER_UID, { admin: true })
  .then(() => console.log('Admin claim set.'));

// middleware.ts -- runs on the server for every /admin request
const decoded = await adminAuth.verifySessionCookie(sessionCookie, true);
if (!decoded.admin) {
  return NextResponse.redirect(new URL('/login', request.url));
}
```

- This claim cannot be set, faked, or escalated from the browser under any circumstance — only code holding the service-account key can call `setCustomUserClaims`.
- A claim change only takes effect on that user's next session cookie — sign out and back in after granting or revoking admin.
- One owner account is enough at launch. Adding a second staff login later is the same script, a different UID, no schema change.

06

Password reset



Firebase Auth's `sendPasswordResetEmail` handles the entire flow — token generation, expiry, and the reset form itself can use Firebase's hosted action page or a custom page on the same domain that reads the `oobCode` from the URL. Either way, the app itself never sees or stores the reset token; Firebase Auth owns that lifecycle end to end.

07

Guest checkout & account linking

An order never requires an account. A guest order is written with `userId: null` and the phone/email captured directly on the order — the Server Action that creates it runs with the Admin SDK, so it is not blocked by a rule requiring a signed-in user.

Deliberately manual, not automatic

When a guest later creates an account, past guest orders are **not** auto-linked by matching phone or email. Automatic linking on an unverified phone number is an account-takeover vector — someone else's old guest order could attach to a new signup. Instead, `/track-order` (order number + phone) always works for guests, and linking a specific order into an account is a deliberate action the signed-in customer takes, verified against the same phone number on the order.

08

Rate limiting & abuse prevention

SURFACE	PROTECTION
Sign-in / sign-up	Firebase Auth's own quota and reCAPTCHA; add Firebase App Check so only the real app's requests are accepted, not a replayed script
/api/assistant	Per-session and per-IP request caps in the route handler — an unmetered AI endpoint is a direct line to an unbounded Anthropic bill
/checkout (order creation)	Per-IP submission cap; server-side validation rejects malformed or duplicate submissions before they reach Firestore
/admin/inventory Excel import	Row-count cap per upload, and every import runs as one atomic batched write that either fully succeeds or fully fails
Contact / custom-order forms	A honeypot field plus a submission cap; these are the routes most likely to attract spam bots

09

Webhook authentication

A webhook is a request from the outside world claiming to be Razorpay or Shiprocket — it is verified, never trusted on its face.

```
// app/api/webhooks/razorpay/route.ts
import crypto from 'crypto';

const signature = request.headers.get('x-razorpay-signature');
const body = await request.text(); // raw body, before any JSON.parse
const expected = crypto
  .createHmac('sha256', process.env.RAZORPAY_WEBHOOK_SECRET!)
  .update(body)
  .digest('hex');

if (signature !== expected) {
  return new Response('Invalid signature', { status: 400 });
}
// only now is the payload trusted and parsed
```

- The raw request body is hashed, not the parsed JSON — Razorpay signs the exact bytes it sent, and re-serializing JSON can change byte-for-byte formatting.
- A webhook is processed **idempotently**: the handler checks whether that event was already applied (e.g. the order is already marked paid) before acting, since a payment gateway can and will redeliver the same webhook.
- Shiprocket's status webhook is checked the same way before it is allowed to move an order to Shipped or Delivered.

10

Where every key actually lives

KEY / CREDENTIAL	IN FRONTEND?	WHY
NEXT_PUBLIC_FIREBASE_* (web config)	Yes	Identifies the project — it is not a secret. Security Rules, not this key's secrecy, are what protect the data
FIREBASE_CLIENT_EMAIL / FIREBASE_PRIVATE_KEY	Never	The Admin SDK service account — bypasses every Security Rule. Server-only, always
RAZORPAY_KEY_ID	Yes	Needed by Razorpay's Checkout.js in the browser; it can only create a payment attempt, not move money on its own
RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET	Never	Can authorize charges and refunds. Server-only, always
SHIPROCKET_EMAIL / SHIPROCKET_PASSWORD	Never	Mints a token with full account access. Server-only, always
ANTHROPIC_API_KEY	Never	Billed per token — an exposed key is a direct line to someone else's usage on your account

The one-sentence version

If it is prefixed NEXT_PUBLIC_, assume every visitor can read it — only ever put genuinely public identifiers there. Everything else is read only inside a Server Action, a Route Handler, or middleware, and is set in Firebase App Hosting's own environment configuration, never committed to git.

11

API & Server Action inventory

ENDPOINT	AUTH REQUIRED	PURPOSE
placeOrder()	None (guest-safe)	Validates cart + address, creates the order, decrements stock
updateStock() / bulkImportStock()	Admin	Powers the inventory table's inline edit and Excel import
createProduct() / updateProduct()	Admin	Product and category CRUD
/api/webhooks/razorpay	Signature-verified	Marks orders paid/failed
/api/webhooks/shiprocket	Signature-verified	Advances order status as a parcel moves
/api/assistant	Rate-limited, public	The AI chat widget — reads the catalog, calls Claude, never writes data

Backend & Authentication Document

How the server side of Raj Raani Collections is put together — the request lifecycle, every layer of the authentication system, and the one rule that keeps every secret out of the browser.

Brand	Raj Raani Collections
Runtime	Next.js 14 on Firebase App Hosting
Golden rule	API keys and secrets live only in server-side environment variables — never in frontend code

01

Backend architecture in one picture

There is no separate backend server to run — Firebase App Hosting runs the Next.js server itself, and that server is the only thing that ever holds a secret.



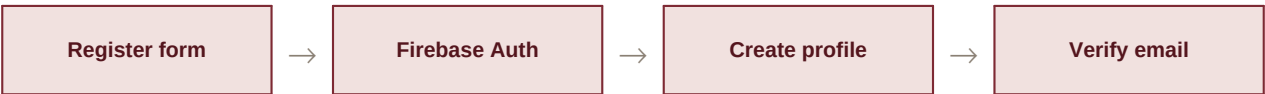
LAYER	RUNS WHERE	JOB
Client components	Browser	UI, forms, the Firebase client SDK (public config only)
Middleware	Server, on every request	Reads the session cookie, verifies it, gates /admin and /account
Server Actions	Server, on demand	Mutations — place an order, edit a product, update stock
Route Handlers	Server, on demand	Webhooks (Razorpay, Shiprocket) and the AI assistant endpoint
Firebase Admin SDK	Server only	The privileged client that bypasses Security Rules — never reaches the browser

The rule that makes this safe

A file with 'use client' at the top runs in the browser — anything referenced there, including any environment variable, ships to every visitor and is readable via view-source. A file without it runs only on the server. Every secret in this document is read only from files in the second category.

02

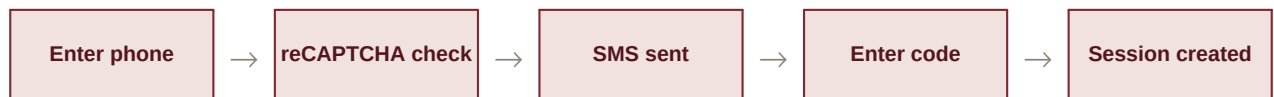
Sign-up — email & password



1. The client calls Firebase Auth's createUserWithEmailAndPassword directly — this is one of the few writes the browser is trusted to make, because Firebase Auth (not Firestore) owns credential storage and password hashing.
2. On success, a Server Action creates the matching users/{uid} profile document — the client never writes that document itself, so a forged profile with a fake role can never be created from the browser.
3. Firebase sends a verification email; the account works immediately, but the app can gate reviews, wishlists shared publicly, or similar trust-sensitive features on emailVerified later if needed.

03

Sign-up & sign-in — phone OTP



1. The client requests a code via Firebase Auth's PhoneAuthProvider, protected by invisible reCAPTCHA (or App Check on supported platforms) so the SMS quota cannot be drained by a script.
2. Firebase sends a 6-digit code by SMS. The customer enters it in the same screen — no separate verify-otp route.
3. `confirmationResult.confirm(code)` completes sign-in entirely inside Firebase Auth; no OTP value or secret ever passes through the app's own server.
4. The same profile-creation Server Action from step 02 runs for a first-time phone sign-in.

Why phone first

Indian shoppers trust an OTP far more than an email link, and it removes "forgot my password" as a support burden entirely for anyone who signs up this way.

04

Sessions — cookies, not raw tokens

The Firebase client SDK issues a short-lived ID token, refreshed automatically in the browser. That token is never the thing middleware checks. Instead:

1. After a successful client-side sign-in, a Server Action calls the Firebase Admin SDK's `createSessionCookie(idToken)`, producing a long-lived, revocable session cookie.
2. That cookie is set **httpOnly, secure, sameSite=lax** — invisible to any JavaScript running on the page, which closes off the most common cookie-theft vector (XSS reading `document.cookie`).
3. Every request to a protected route runs through **middleware**, which calls `getAuth().verifySessionCookie(cookie, checkRevoked=true)` on the server — a valid-looking cookie that has been revoked is rejected, not just an expired one.
4. Sign-out clears the cookie **and** the client SDK session. For a compromised account, calling `revokeRefreshTokens(uid)` invalidates every session for that user immediately, everywhere, not just on the device that reported the problem.

05

The admin role

Admin is not a field a document can carry — it is a **custom claim** baked into the session cookie itself, set once via the Admin SDK and checked on every request.

```
// scripts/make-admin.js -- run once, from a trusted machine, then remove or lock down
const admin = require('firebase-admin');
admin.initializeApp({ credential:
  admin.credential.cert(require('./service-account.json')) });

admin.auth().setCustomUserClaims(OWNER_UID, { admin: true })
  .then(() => console.log('Admin claim set.'));

// middleware.ts -- runs on the server for every /admin request
const decoded = await adminAuth.verifySessionCookie(sessionCookie, true);
if (!decoded.admin) {
  return NextResponse.redirect(new URL('/login', request.url));
}
```

- This claim cannot be set, faked, or escalated from the browser under any circumstance — only code holding the service-account key can call `setCustomUserClaims`.
- A claim change only takes effect on that user's next session cookie — sign out and back in after granting or revoking admin.
- One owner account is enough at launch. Adding a second staff login later is the same script, a different UID, no schema change.

06

Password reset



Firebase Auth's `sendPasswordResetEmail` handles the entire flow — token generation, expiry, and the reset form itself can use Firebase's hosted action page or a custom page on the same domain that reads the `oobCode` from the URL. Either way, the app itself never sees or stores the reset token; Firebase Auth owns that lifecycle end to end.

07

Guest checkout & account linking

An order never requires an account. A guest order is written with `userId: null` and the phone/email captured directly on the order — the Server Action that creates it runs with the Admin SDK, so it is not blocked by a rule requiring a signed-in user.

Deliberately manual, not automatic

When a guest later creates an account, past guest orders are **not** auto-linked by matching phone or email. Automatic linking on an unverified phone number is an account-takeover vector — someone else's old guest order could attach to a new signup. Instead, `/track-order` (order number + phone) always works for guests, and linking a specific order into an account is a deliberate action the signed-in customer takes, verified against the same phone number on the order.

08

Rate limiting & abuse prevention

SURFACE	PROTECTION
Sign-in / sign-up	Firebase Auth's own quota and reCAPTCHA; add Firebase App Check so only the real app's requests are accepted, not a replayed script
/api/assistant	Per-session and per-IP request caps in the route handler — an unmetered AI endpoint is a direct line to an unbounded Anthropic bill
/checkout (order creation)	Per-IP submission cap; server-side validation rejects malformed or duplicate submissions before they reach Firestore
/admin/inventory Excel import	Row-count cap per upload, and every import runs as one atomic batched write that either fully succeeds or fully fails
Contact / custom-order forms	A honeypot field plus a submission cap; these are the routes most likely to attract spam bots

09

Webhook authentication

A webhook is a request from the outside world claiming to be Razorpay or Shiprocket — it is verified, never trusted on its face.

```
// app/api/webhooks/razorpay/route.ts
import crypto from 'crypto';

const signature = request.headers.get('x-razorpay-signature');
const body = await request.text(); // raw body, before any JSON.parse
const expected = crypto
  .createHmac('sha256', process.env.RAZORPAY_WEBHOOK_SECRET!)
  .update(body)
  .digest('hex');

if (signature !== expected) {
  return new Response('Invalid signature', { status: 400 });
}
// only now is the payload trusted and parsed
```

- The raw request body is hashed, not the parsed JSON — Razorpay signs the exact bytes it sent, and re-serializing JSON can change byte-for-byte formatting.
- A webhook is processed **idempotently**: the handler checks whether that event was already applied (e.g. the order is already marked paid) before acting, since a payment gateway can and will redeliver the same webhook.
- Shiprocket's status webhook is checked the same way before it is allowed to move an order to Shipped or Delivered.

10

Where every key actually lives

KEY / CREDENTIAL	IN FRONTEND?	WHY
NEXT_PUBLIC_FIREBASE_* (web config)	Yes	Identifies the project — it is not a secret. Security Rules, not this key's secrecy, are what protect the data
FIREBASE_CLIENT_EMAIL / FIREBASE_PRIVATE_KEY	Never	The Admin SDK service account — bypasses every Security Rule. Server-only, always
RAZORPAY_KEY_ID	Yes	Needed by Razorpay's Checkout.js in the browser; it can only create a payment attempt, not move money on its own
RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET	Never	Can authorize charges and refunds. Server-only, always
SHIPROCKET_EMAIL / SHIPROCKET_PASSWORD	Never	Mints a token with full account access. Server-only, always
ANTHROPIC_API_KEY	Never	Billed per token — an exposed key is a direct line to someone else's usage on your account

The one-sentence version

If it is prefixed NEXT_PUBLIC_, assume every visitor can read it — only ever put genuinely public identifiers there. Everything else is read only inside a Server Action, a Route Handler, or middleware, and is set in Firebase App Hosting's own environment configuration, never committed to git.

11

API & Server Action inventory

ENDPOINT	AUTH REQUIRED	PURPOSE
placeOrder()	None (guest-safe)	Validates cart + address, creates the order, decrements stock
updateStock() / bulkImportStock()	Admin	Powers the inventory table's inline edit and Excel import
createProduct() / updateProduct()	Admin	Product and category CRUD
/api/webhooks/razorpay	Signature-verified	Marks orders paid/failed
/api/webhooks/shiprocket	Signature-verified	Advances order status as a parcel moves
/api/assistant	Rate-limited, public	The AI chat widget — reads the catalog, calls Claude, never writes data